

Generador de Logs Crudos Masivos

Documento guía para la Fase 1 del Proyecto Transversal

Asignatura: Programación Avanzada

Alcance: Semanas 3 a 5. Defensa del entregable en Semana 5.

Caso de referencia: Simulador Logístico del Terminal Puerto Coquimbo (TPC), Sitio 3.

Antes de empezar

Este documento no es un enunciado cerrado. Es una guía de arquitectura: reúne las decisiones técnicas que sabemos que funcionan y los errores de concurrencia que son clásicos en este tipo de problema. Léalo completo antes de escribir la primera línea de código: la mitad de los problemas que aparecen en la Semana 4 se originan en decisiones tomadas apurados el primer día.

Hay dos arquitecturas válidas para resolver el problema, y ambas están descritas aquí. Elegir una y defenderla con datos vale más que intentar las dos a medias. Lo que sí importa es que la decisión esté justificada con mediciones propias, no con lo que dice este documento ni con lo que responde un modelo de lenguaje.

Cualquier duda de interpretación se resuelve en clases o por el canal de la asignatura, y conviene hacerlo apenas aparezca, no el domingo previo a la entrega.

1. Introducción y contexto del desafío

1.1 El reto de la escala

El Terminal Puerto Coquimbo mueve carga general, contenedores refrigerados y concentrado de mineral. Para efectos del proyecto, ustedes van a modelar el Sitio 3: un frente de atraque con fajas transportadoras, dos grúas móviles, una báscula de acceso vehicular y un patio con contenedores reefer conectados a la red eléctrica del terminal.

La instrucción es simular el comportamiento físico de ese entorno, y la palabra importante es física. No estamos pidiendo un generador de números aleatorios con nombres bonitos. Un contenedor refrigerado con setpoint en -18°C no salta a $+4^{\circ}\text{C}$ entre dos lecturas separadas por treinta segundos, salvo que alguien haya desconectado el equipo. Si su generador produce esa secuencia, en la Fase 2 el detector de anomalías la marcará como alarma crítica y ustedes habrán fabricado un falso positivo perfecto, imposible de distinguir de una falla real.

La diferencia práctica entre un dataset creíble y uno inservible es el estado. Cada activo simulado debe recordar en qué condición quedó en el tick anterior:

```
class Reefer:
    def __init__(self, sid, rng, setpoint=-18.0):
        self.sid = sid
        self.rng = rng
        self.setpoint = setpoint
        self.temp = setpoint + rng.uniform(-0.4, 0.4)
        self.puerta_abierta = False

    def tick(self, dt_s):
        # el compresor tira la temperatura hacia el setpoint;
```

```
# la puerta abierta la empuja en sentido contrario
if self.puerta_abierta:
    self.temp += 0.9 * (dt_s / 60.0)
else:
    self.temp += (self.setpoint - self.temp) * 0.08
self.temp += self.rng.gauss(0.0, 0.05) # ruido del sensor
return round(self.temp, 2)
```

Diecisiete líneas. Eso basta para que la serie temporal tenga inercia, deriva y ruido, que es exactamente lo que la Fase 2 necesita para ser interesante. Repliquen la misma idea para la vibración de una faja (sube con la carga, cae cuando se detiene el flujo), para el amperaje del motor y para el peso registrado en la báscula.

Sugerencia de inventario para el Sitio 3: 48 posiciones reefer, 6 tramos de faja con sensores de vibración y corriente, 2 grúas con telemetría de ciclo, 1 báscula de acceso y 12 sensores ambientales de patio. Alrededor de 180 puntos de medición reportando cada 15 segundos.

1.2 La meta de volumen

El simulador debe producir un mínimo de 10 millones de eventos. No es una cifra arbitraria: con los 180 puntos del párrafo anterior a un evento cada 15 segundos, el terminal genera algo más de un millón de registros diarios, así que 10 millones equivalen a poco más de nueve días de operación continua. La gracia del ejercicio es que esos nueve días deben materializarse en minutos, no en días.

Traduzcan eso a magnitudes concretas antes de programar. Un evento JSON del esquema mínimo pesa entre 130 y 170 bytes según la longitud de los identificadores. Diez millones de eventos son, entonces, del orden de 1,4 a 1,7 GB en disco. Ese número manda sobre todo lo demás: define cuánto espacio necesitan libre, cuánto va a demorar la escritura y por qué está terminantemente prohibido subir el dataset a Git.

Como referencia gruesa, y solo como referencia, un único proceso de CPython bien escrito genera y serializa entre 80.000 y 150.000 eventos por segundo en un equipo de estudiante. Es decir, entre 70 y 125 segundos para la meta completa. Con paralelismo real esperamos ver eso caer a un rango de 15 a 30 segundos. Si sus números están muy por debajo, hay un cuello de botella que encontrar; si están muy por encima, revisen que efectivamente estén escribiendo todo lo que dicen escribir.

1.3 Por qué los hilos no sirven aquí

La pregunta es razonable y hay que responderla bien: si el problema es hacer muchas cosas a la vez, ¿por qué no usar threading, que es más simple y comparte memoria? La respuesta corta es que en CPython no van a ganar nada. La respuesta larga vale la pena.

CPython protege sus estructuras internas con el Global Interpreter Lock. Solo un hilo ejecuta bytecode a la vez dentro de un mismo intérprete. El GIL se libera cuando un hilo se bloquea esperando entrada/salida, y por eso threading brilla en clientes HTTP o lectura de archivos. Pero miren qué hace realmente su generador en cada iteración: llama a random, actualiza el estado del activo, formatea una marca de tiempo y serializa un diccionario a JSON. Todo eso es aritmética y manipulación de objetos, o sea bytecode puro. El trabajo es CPU-bound, y contra el GIL eso significa que ocho hilos se turnan el mismo núcleo.

Peor todavía: agregan el costo de los cambios de contexto y la contención por el propio candado. Es habitual que la versión con cuatro hilos resulte algo más lenta que la versión secuencial. Médanlo ustedes, es un experimento de diez minutos y es el mejor argumento que van a tener en la defensa.

La salida es multiprocessing. Cada proceso hijo es un intérprete independiente, con su propio GIL y su propio espacio de memoria, ejecutándose en un núcleo distinto. El paralelismo pasa a ser real. A cambio se pagan

tres peajes que van a ver aparecer una y otra vez en este documento: crear procesos cuesta tiempo, no comparten memoria, y todo dato que cruce entre ellos debe serializarse con pickle.

Sobre el build sin GIL de Python 3.13 y 3.14: existe, funciona y es el futuro. En este proyecto no se usa. El objetivo pedagógico de la Fase 1 es que dominen el modelo de procesos y sus costos de comunicación, que es lo que van a encontrar en Spark, Celery, Dask y en cualquier motor distribuido serio. Si les interesa el tema, es material excelente para la sección de trabajo futuro del informe.

Trampa de ingeniería N.º 1. Medir la ganancia de los hilos con un caso de prueba de mil eventos. A esa escala el ruido del sistema operativo domina cualquier medición y van a ver lo que quieran ver. Los efectos del GIL, igual que los de corrupción de archivos, aparecen recién cuando el volumen es grande y sostenido. Ningún experimento de este proyecto se valida con menos de un millón de eventos.

2. Arquitectura del generador y del almacenamiento

2.1 El problema real: escrituras concurrentes

Resuelto el paralelismo del cálculo, aparece el problema que de verdad define la arquitectura. Ocho procesos que generan eventos en paralelo tienen que dejarlos en alguna parte, y la primera idea que se le ocurre a cualquiera es esta:

```
# ANTIPATRÓN. No hagan esto.
def worker_inseguro(n_eventos, ruta):
    with open(ruta, "a", encoding="utf-8") as f:      # <-- el mismo archivo
        for _ in range(n_eventos):
            f.write(json.dumps(generar_evento()) + "\n")

with ProcessPoolExecutor(max_workers=8) as pool:
    for _ in range(8):
        pool.submit(worker_inseguro, 1_250_000, "data/raw/eventos.jsonl")
```

Esto parece funcionar. Corre sin lanzar excepciones, el archivo crece, `wc -l` devuelve un número plausible. Y el dataset está roto.

La razón es el búfer. Cada proceso tiene su propio búfer de usuario, típicamente de 8 KB, y lo vacía al disco cuando se llena, en un momento que ningún proceso controla y que no coincide con el final de una línea. El resultado son líneas cortadas por la mitad y luego pegadas al fragmento de otro proceso. En el archivo final van a encontrar cosas como:

```
{"timestamp": "2026-08-24T18:40:41.123Z", "sensor_id": "REEF{"timestamp": "2026-08-
24T18:40:41.187Z", ...}
ER_S3_04", "metric": "temperature", "value": -18.4, "unit": "C", "worker_id": 3}
```

Dos registros destruidos y, lo que es peor, dos líneas que no parsean. Spark las va a descartar en modo permisivo o va a caer el job entero en modo estricto, y ustedes lo van a descubrir en la Semana 8 sin entender por qué. Sobre `O_APPEND` en Linux hay literatura que promete atomicidad, pero solo bajo condiciones de tamaño que su búfer no respeta y con comportamiento distinto en NTFS. Es una garantía que no se puede usar.

Existen exactamente dos formas correctas de salir de esto, y las dos son aceptables para la entrega.

2.2 Opción A: proceso escritor único

Se separa el rol de calcular del rol de escribir. Los trabajadores hacen la física y la serialización; un proceso dedicado, el listener, es el único que tiene el archivo abierto y escribe de manera secuencial. La comunicación va por una cola multiproceso, que es segura por construcción.

```
import json, random
import multiprocessing as mp

CENTINELA = "__FIN__"
LOTE = 5_000 # eventos por mensaje: amortiza el costo de pickle

def escritor(cola, ruta, n_workers):
    vivos = n_workers
    with open(ruta, "w", encoding="utf-8", newline="\n",
              buffering=1024*1024) as f:
        while vivos:
            lote = cola.get() # bloquea si la cola está vacía
            if lote == CENTINELA:
                vivos -= 1 # un worker menos, seguimos con el resto
                continue
            f.writelines(lote) # llega ya serializado, sin trabajo extra

def trabajador(cola, wid, n_eventos, semilla):
    rng = random.Random(semilla + wid) # semilla propia por proceso
    buf = []
    for _ in range(n_eventos):
        ev = generar_evento(rng, wid)
        buf.append(json.dumps(ev, separators=(",", ":")) + "\n")
        if len(buf) >= LOTE:
            cola.put(buf)
            buf = []
    if buf:
        cola.put(buf)
    cola.put(CENTINELA) # aviso de término

if __name__ == "__main__":
    cola = mp.Queue(maxsize=64) # imprescindible, ver trampa N.º 3
    # contrapresión: NUNCA ilimitada
    ...
```

Tres detalles de ese código que no son cosméticos. Primero, los trabajadores serializan a texto antes de encolar, de modo que el escritor solo hace writelines y nunca se transforma en el cuello de botella. Segundo, se envían lotes de miles de eventos y no eventos sueltos: cada put implica un pickle, un write a un pipe y un unpickle del otro lado, y a razón de diez millones de operaciones ese costo se come toda la ganancia del paralelismo. Tercero, el centinela es cómo el escritor sabe que puede cerrar el archivo; sin él, el proceso queda esperando para siempre y el programa nunca termina.

Advertencia sobre memoria. Este patrón tiene un modo de falla propio y bastante brutal. Si los ocho trabajadores producen a 600.000 eventos por segundo y el disco sostiene 200.000, la diferencia no desaparece: se acumula en el búfer de la cola, que vive en RAM. Son 400.000 eventos por segundo a unos 150 bytes cada uno, o sea 60 MB por segundo creciendo. Al minuto de ejecución llevan 3,6 GB y el equipo empieza a hacer swap; poco después el kernel mata el proceso. Por eso la cola se crea con maxsize acotado. Cuando se llena, put bloquea, los trabajadores se frenan solos y el sistema se autorregula al ritmo del disco. Esa contrapresión no es una limitación, es la característica que hace utilizable el patrón.

Trampa de ingeniería N.º 2. Usar queue.Queue en lugar de multiprocessing.Queue. La primera es para hilos dentro de un proceso y no cruza el límite entre procesos: al pasarla como argumento, cada hijo recibe una copia vacía suya y escribe en el vacío. No hay error, no hay excepción, simplemente el archivo final queda con cero líneas y el programa termina feliz. Si su salida está vacía o llena solo a medias, este es el primer lugar donde mirar.

2.3 Opción B: archivos segmentados por proceso

La alternativa elimina el problema en vez de gestionarlo. Si cada trabajador escribe en su propio archivo, no hay recurso compartido, no hay cola, no hay serialización entre procesos y no hay nada que sincronizar.

```
from pathlib import Path

def trabajador_shard(wid, n_eventos, carpeta, semilla):
    rng = random.Random(semilla + wid)
    ruta = Path(carpeta) / f"part-{wid:03d}.jsonl"
    escritos = 0
    buf = []
    with open(ruta, "w", encoding="utf-8", newline="\n",
              buffering=4*1024*1024) as f:
        for _ in range(n_eventos):
            ev = generar_evento(rng, wid)
            buf.append(json.dumps(ev, separators=(",", ":")) + "\n")
            if len(buf) == 10_000:
                f.writelines(buf)
                escritos += len(buf)
                buf = []
        f.writelines(buf)
        escritos += len(buf)
    # lo único que vuelve al proceso padre son estadísticas, no datos
    return {"worker_id": wid, "archivo": ruta.name,
            "eventos": escritos, "bytes": ruta.stat().st_size}
```

El proceso maestro recoge esos diccionarios, los suma y escribe un manifiesto con el total de eventos, el tamaño en disco, la semilla usada, el número de trabajadores y el tiempo total. Ese manifiesto vale oro para el informe de benchmarking y para verificar que no se perdió nada.

La ventaja es que no hay colisiones de entrada/salida a nivel de software y el sistema operativo puede planificar las escrituras de los ocho descriptores como le convenga, aprovechando el ancho de banda completo del disco. En SSD NVMe la diferencia frente a la Opción A suele ser notoria. En discos mecánicos puede ser al revés, porque ocho flujos simultáneos obligan al cabezal a saltar; si alguien todavía trabaja sobre HDD, tiene ahí un hallazgo excelente para el informe.

Trampa de ingeniería N.º 3. El olvido del bloque `if __name__ == "__main__"`. En Linux, multiprocessing usa fork por defecto y el código funciona igual. En Windows y en macOS moderno el método por defecto es spawn: el proceso hijo vuelve a importar el módulo completo, y si el código que lanza el pool está al nivel superior del archivo, cada hijo lanza su propio pool. El resultado es una bomba de procesos que congela el equipo. Prueben su generador con `mp.set_start_method("spawn")` aunque trabajen en Linux, porque la corrección puede hacerse en otra plataforma.

Trampa de ingeniería N.º 4. Sembrar el generador aleatorio antes del fork. Con fork, todos los hijos heredan el estado exacto del módulo random del padre. Si no le dan una semilla propia a cada proceso, los ocho trabajadores generan la misma secuencia de valores y el dataset queda con ocho copias idénticas de la misma serie. Es un error que no rompe nada, no lanza advertencias y solo se nota si alguien revisa los datos. Nosotros los revisamos.

La solución es la de los ejemplos anteriores: `random.Random(semilla + worker_id)` instanciado dentro del proceso hijo, nunca el módulo global.

2.4 Cómo elegir

Aspecto	Opción A: escritor único	Opción B: archivos segmentados
Complejidad de implementación	Alta. Cola, centinelas, cierre ordenado, manejo de bloqueos.	Baja. Cada proceso es independiente y el maestro solo agrega.
Costo de comunicación	Un pickle por lote más el paso por el pipe del sistema.	Prácticamente nulo. Solo vuelven estadísticas al final.

Aspecto	Opción A: escritor único	Opción B: archivos segmentados
Riesgo dominante	Desbordamiento de memoria si falta contrapresión.	Proliferación de archivos pequeños y saturación del disco.
Salida	Un archivo único y ordenado.	N archivos, uno por trabajador.
Rendimiento esperado	Bueno, con techo en la velocidad del proceso escritor.	Mayor, limitado por el disco y no por el software.

Recomendación honesta: si su equipo tiene poca experiencia con multiprocessing, implementen la Opción B, que tiene menos formas de fallar, y dediquen el tiempo ahorrado a la calidad de la simulación y del informe. Si quieren aspirar a la nota máxima en la sección arquitectónica, implementen ambas detrás de la misma interfaz y compárenlas con datos. Un argumento del tipo «elegimos B porque a partir de cuatro procesos el escritor único saturaba y la cola acumulaba 1,2 GB» vale muchísimo más que cualquier explicación teórica.

Trampa de ingeniería N.º 5. Concatenar los fragmentos al final para «dejarlo ordenado en un solo archivo». Si eligieron la Opción B por velocidad y después leen y reescriben 1,5 GB con cat, acaban de regalar la ventaja completa. Spark lee una carpeta de forma nativa, y de hecho prefiere varios archivos porque los reparte entre sus ejecutores. Consoliden solo si tienen una razón concreta, e inclúyanla en el informe.

3. Herramientas y especificaciones de logging

3.1 Por qué JSON Lines y no otra cosa

El formato obligatorio es JSON Lines, extensión .jsonl. Un objeto JSON completo por línea, sin corchetes envolventes, sin comas al final, sin saltos de línea dentro del objeto. La decisión no es estética y conviene entender los tres descartes.

El texto plano desestructurado se descarta porque obliga a escribir un parser. En cuanto un campo de texto contenga el separador que eligieron, el parser se rompe, y toda la Fase 2 pasa a depender de expresiones regulares frágiles en vez de análisis de datos.

XML se descarta por volumen y por costo de lectura. Las etiquetas de apertura y cierre multiplican el tamaño del archivo por tres o cuatro, y su lectura eficiente requiere procesamiento por streaming que ni ustedes ni Spark van a querer configurar para este caso.

Un archivo JSON convencional, con un arreglo grande de objetos, se descarta por una razón más sutil y más importante: no es divisible. Para saber dónde termina el objeto número 4.000.000 hay que haber leído todo lo anterior, así que el archivo completo debe ser procesado por un solo hilo y cargado entero en memoria. Un .jsonl es divisible por definición, porque el salto de línea es un delimitador que se puede encontrar desde cualquier posición del archivo. Spark parte el archivo en bloques, los reparte entre ejecutores y los procesa en paralelo. Es la diferencia entre un análisis que corre en un minuto y uno que revienta por falta de memoria.

En la Fase 2, con el formato correcto, la lectura completa del dataset es una línea:

```
df = spark.read.json("data/raw/") # lee la carpeta entera, infiere el esquema
df.printSchema()
df.count()
```

Trampa de ingeniería N.º 6. Comprimir la salida con gzip para ahorrar espacio. gzip tampoco es divisible: Spark tendría que descomprimir cada archivo con un solo núcleo antes de poder repartir el trabajo. Para esta fase la salida se entrega sin comprimir. Si el espacio en disco es un problema real, consúltenlo antes de improvisar una solución.

3.2 Esquema mínimo del evento

Todos los grupos comparten el mismo contrato de datos. En la Fase 2 vamos a correr consultas de referencia sobre sus datasets, y si cada equipo inventa sus nombres de campo eso se vuelve imposible. Estos seis campos son obligatorios y sus nombres son literales:

```
{"timestamp": "2026-08-24T18:40:41.123Z", "sensor_id": "REEFER_S3_04",  
  "metric": "temperature", "value": -18.4, "unit": "C", "worker_id": 3}
```

En el archivo real todo eso va en una sola línea; arriba está partido en dos únicamente para que quepa en la página.

- **timestamp:** cadena ISO 8601 en UTC, con milisegundos y sufijo Z. Nada de horas locales ni de objetos datetime sin zona horaria.
- **sensor_id:** identificador estable con la convención TIPO_SITIO_CORRELATIVO. Por ejemplo REEFER_S3_04, FAJA_S3_C2, GRUA_S3_STS01, BASCULA_S3_01.
- **metric:** nombre de la magnitud en minúsculas y en inglés. temperature, vibration, current, weight, humidity.
- **value:** numérico, no cadena. Escribir "-18.4" entre comillas obliga a castear tres millones de filas en Spark.
- **unit:** unidad física del valor. C, mm/s, A, kg, %.
- **worker_id:** entero del proceso que generó el evento. Es el campo que hace auditable el paralelismo, porque permite verificar el balance de carga contando eventos por worker.

Campos opcionales que recomendamos y que van a agradecer en la Fase 2: site para el sitio del terminal, asset_type para agrupar por familia de activo, status con valores OK, WARN o ALARM, y seq como correlativo por sensor para detectar huecos en la serie. Si los agregan, agréguenlos en todos los eventos: un esquema irregular obliga a Spark a inferir tipos nulos y complica las consultas.

Trampa de ingeniería N.º 7. Llamar a `datetime.now(timezone.utc).isoformat()` diez millones de veces. Esa llamada, más el formateo de la cadena, puede representar un tercio del tiempo total de ejecución. La medida rápida es calcular una marca base al inicio del bloque y avanzarla con aritmética entera de milisegundos, formateando solo lo necesario. Midan antes y después con cProfile: es el tipo de optimización que luce muy bien en el informe porque está respaldada con números.

3.3 Librerías permitidas y prohibidas

El generador se escribe exclusivamente con la biblioteca estándar de Python 3.11 o superior. Están habilitados multiprocessing, concurrent.futures, json, logging, argparse, random, datetime, time, os, pathlib, statistics, cProfile y resource o tracemalloc para las mediciones de memoria.

Quedan prohibidos dentro del generador pandas, numpy, polars, faker, loguru, joblib, ray, dask, orjson y ujson. No es porque sean malas, es porque son buenas: numpy resuelve el problema de generación en cuatro líneas vectorizadas y orjson triplica la velocidad de serialización, y en ambos casos ustedes se habrían saltado justamente lo que la asignatura quiere que aprendan. La restricción aplica al generador, no al análisis.

Excepción explícita: para producir los gráficos del informe de benchmarking pueden usar matplotlib o la herramienta que prefieran, incluida una planilla de cálculo. Ese código va en /bench/ o fuera del repositorio y no forma parte del simulador.

3.4 Una advertencia sobre el módulo logging

El módulo logging aparece en la lista de permitidos y eso genera una confusión que conviene despejar. logging sirve para el registro operativo del simulador: avance de la generación, advertencias, excepciones capturadas, resumen final. No sirve para emitir los diez millones de eventos.

Cada llamada a logger.info adquiere un candado, arma un objeto LogRecord, recorre la cadena de handlers y aplica el formateo. Eso es entre tres y cinco veces más lento que un writelines sobre un búfer grande. Y con multiprocessing hay un problema adicional: los handlers no se comparten entre procesos, así que un FileHandler abierto por el padre y heredado por ocho hijos reproduce exactamente el problema de corrupción de la sección 2.1, ahora escondido detrás de una API que parece confiable.

La regla es simple. Los eventos de la simulación se escriben con open y writelines. El log operativo del proceso usa logging, y si quieren hacerlo bien con varios procesos, la forma correcta es QueueHandler en los hijos y QueueListener en el padre. Vale como punto extra en la defensa, no como requisito.

4. Entregable de la Semana 5 y criterios de evaluación

4.1 Estructura del repositorio

Se entrega un repositorio Git, no un archivo comprimido por correo. La estructura mínima esperada es la siguiente:

```
proyecto-tpc-fase1/
src/
  main.py           # punto de entrada, argparse
  simulador.py      # modelos de los activos y su física
  arquitectura_a.py # patrón escritor único
  arquitectura_b.py # patrón archivos segmentados
  esquema.py        # construcción y validación del evento
data/
  raw/              # salida .jsonl (IGNORADA por git)
  muestra/           # 50.000 eventos de ejemplo, esto SÍ se versiona
bench/
  mediciones.csv    # datos crudos de todas las corridas
  graficos.py
docs/
  reporte_benchmarking.pdf
README.md
.gitignore
```

4.2 Código fuente parametrizable

El simulador debe ser configurable por línea de comandos con argparse. Sin valores mágicos escondidos en el código: si hay que abrir y editar un archivo para cambiar la cantidad de procesos, el requisito no está cumplido. Como mínimo:

```
python -m src.main \
  --workers 8 \
  --total-events 10000000 \
  --output-dir data/raw \
  --arch shard \           # queue | shard
  --batch-size 10000 \
  --seed 42 \
```


Dos exigencias concretas sobre esto. La primera: con la misma semilla y el mismo número de trabajadores, dos ejecuciones deben producir datasets idénticos. Es lo que permite depurar y lo que permite que nosotros verifiquemos. La segunda: el programa debe validar sus argumentos y fallar temprano con un mensaje claro. Pedir --workers 200 en un equipo de ocho núcleos debería advertir, no quedar colgado veinte minutos.

El README explica cómo instalar, cómo ejecutar y cuánto demora y cuánto ocupa. Tres párrafos bien escritos bastan.

4.3 Dataset de logs crudos

La carpeta data/raw/ debe contener los archivos .jsonl con al menos 10 millones de registros generados por su simulador, más un manifiesto en JSON con la metadata de la corrida: total de eventos, archivos producidos, bytes escritos, semilla, número de trabajadores, arquitectura usada y duración.

Ese dataset no se sube a Git. Son más de 1,4 GB y GitHub rechaza archivos sobre 100 MB, además de que el historial del repositorio quedaría inutilizable para siempre. Incluyan data/raw/ en el .gitignore desde el primer commit, versionen una muestra de 50.000 eventos en data/muestra/ y dejen en el README el comando exacto que regenera el dataset completo. Al momento de la defensa, el dataset íntegro debe estar en el disco del equipo con el que presentan.

Verifiquen antes de entregar. Cuesta dos minutos y evita el peor escenario posible, que es descubrir durante la corrección que el dataset no parsea:

```
wc -l data/raw/*.jsonl      # ¿suman 10 millones de líneas?
du -sh data/raw/           # ¿el tamaño calza con lo esperado?
```

Y una validación que recorra el archivo completo, no las primeras líneas. Los registros rotos por concurrencia aparecen en el medio, nunca al principio:

```
# bench/validar.py
import glob, json

total = 0
for ruta in sorted(glob.glob("data/raw/*.jsonl")):
    with open(ruta, encoding="utf-8") as f:
        for i, linea in enumerate(f, 1):
            try:
                json.loads(linea)
            except json.JSONDecodeError:
                raise SystemExit(f"Línea inválida en {ruta}:{i}")
            total += 1
print("OK,", total, "eventos válidos")
```

4.4 Reporte técnico de benchmarking

Un PDF de entre seis y doce páginas, escrito como informe de ingeniería y no como bitácora de tareas. Debe contener lo siguiente.

1. Descripción del entorno de pruebas: procesador, número de núcleos físicos y lógicos, memoria RAM, tipo de disco (esto importa mucho), sistema operativo y versión exacta de Python. Sin esto, ninguna medición es interpretable.
2. Curva de escalabilidad con 1, 2, 4, 8 y 16 trabajadores, manteniendo fijo el total de eventos. Cada configuración se ejecuta tres veces y se reporta la mediana, no la mejor corrida. La primera ejecución siempre es más lenta por el caché de páginas del sistema operativo, así que descarten una corrida previa de calentamiento y déjenlo dicho.

3. Speedup y eficiencia. $S(n) = T(1) / T(n)$, y $E(n) = S(n) / n$. Grafiquen la curva junto a la recta del speedup ideal. La distancia entre ambas es el tema central del informe.
4. Rendimiento de entrada/salida: eventos por segundo y MB/s efectivos escritos, por cada configuración.
5. Análisis del costo de pickle. Si implementaron la Opción A, midan el mismo experimento con lotes de 1, 100, 1.000 y 10.000 eventos por put. La diferencia es dramática y explica por sí sola por qué la comunicación entre procesos define la arquitectura.
6. Gestión de memoria: consumo máximo observado, qué maxsize eligieron para la cola y por qué, y si llegaron a provocar un desbordamiento durante las pruebas. Si lo provocaron, cuéntenlo. Es un resultado.
7. Interpretación crítica. Aquí se juega la nota. ¿Dónde se aplana la curva y por qué? Candidatos: saturación del disco, hyperthreading que no aporta en cargas de CPU, costo de creación de procesos con spawn, o la fracción secuencial del programa según la ley de Amdahl. Nombren la causa y justifiquenla con sus propias mediciones.

Sobre la honestidad de los resultados, conviene dejarlo dicho desde ahora: si con 16 procesos su simulador rinde peor que con 8, repórtenlo tal cual. Eso es exactamente lo que esperamos que ocurra en un equipo de 8 núcleos, y explicar por qué ocurre demuestra más comprensión que una curva perfecta y sospechosa. Los datos crudos de todas las corridas van en `bench/mediciones.csv`.

Trampa de ingeniería N.º 8. Presentar como evidencia capturas de pantalla del terminal. Un informe de ingeniería se sustenta en datos tabulados y gráficos con ejes rotulados y unidades. Instrumenten el simulador para que emita sus propias métricas en CSV al terminar cada corrida, y construyan los gráficos desde ahí. Toma menos tiempo que recortar imágenes y se puede rehacer entero cuando cambien algo.

4.5 Defensa oral

Doce minutos de presentación y ocho de preguntas, en la Semana 5. Preguntamos a cualquier integrante sobre cualquier parte del código, así que el reparto de tareas no exime a nadie de entender el conjunto. Para que se preparen, este es el tipo de pregunta que van a recibir:

- Muéstrenme en el código el punto exacto donde se paga el costo de pickle.
- Si les quito la mitad de los núcleos, ¿qué número cambian y qué esperan que pase con el tiempo total?
- ¿Qué ocurre si el proceso escritor muere a mitad de la simulación? ¿Cómo se enteran?
- ¿Por qué su curva de speedup se aplana en ese punto y no antes?
- Abran un archivo del dataset en la última línea y explíquenme el evento que aparece ahí.

4.6 Rúbrica

Criterio	Qué se evalúa	Peso
Arquitectura y corrección concurrente	Elección justificada del patrón, ausencia de corrupción en el dataset, cierre ordenado de procesos y archivos, manejo de errores.	30 %

Criterio	Qué se evalúa	Peso
Calidad del dataset	Volumen mínimo alcanzado, adherencia estricta al esquema, coherencia física de las series temporales, validez de todas las líneas.	20 %
Reporte de benchmarking	Rigor experimental, correcta construcción de los gráficos, profundidad del análisis crítico, tratamiento del costo de serialización y de la memoria.	30 %
Calidad del código	Parametrización con argparse, modularidad, legibilidad, documentación, reproducibilidad con semilla.	10 %
Defensa oral	Dominio individual del trabajo del equipo, capacidad de responder sobre decisiones de diseño.	10 %

Un dataset con líneas corruptas o con menos de 10 millones de registros deja el criterio de calidad del dataset en cero, independientemente de lo elegante que sea el código. Es el requisito duro de la fase.

4.7 Ritmo sugerido

Semana 3: modelo de los activos y su física, esquema del evento, generador secuencial que produzca cien mil eventos correctos. Al final de la semana ya deberían tener el experimento de hilos contra procesos medido.

Semana 4: paralelización con la arquitectura elegida, parametrización completa y primera corrida de diez millones. Aquí es donde aparecen la memoria y las trampas del arranque de procesos. Reserven tiempo.

Semana 5: campaña de mediciones, redacción del informe y ensayo de la defensa. La campaña completa toma varias horas de máquina; empezarla el día anterior no alcanza.

Cierre

Al terminar esta fase van a tener un dataset propio de más de un gigabyte, generado por código que ustedes escribieron y entienden, con una explicación medida de por qué corre a la velocidad que corre. Eso es material de portafolio, y es también el insumo directo de la Fase 2: sin un dataset limpio y bien formado, el trabajo con Spark se convierte en limpieza de datos y no en análisis.

Consultas técnicas en clases o por el canal de la asignatura. Si el problema es de concurrencia, traigan el código y la medición, no la descripción del síntoma.